

EECS 4080 SU26

Project title: Detecting direct prompt injection attacks in LLM applications

Supervisor: Professor Sukhwant Kaur Sagar

Student: Thien Nguyen

Midterm progress report & Milestone 3 revision summary

Gaps and Required Actions

1. F1 Score Below Contract Target — Acknowledge and Contextualize

Your contract specifies an $F1 \geq 85\%$ for the primary DistilBERT model. Your test set result of 83.31% is 1.69 percentage points below this threshold. This is not a large gap, and there is a defensible explanation — your dataset is dominated by hard jailbreak-style adversarial examples that even the industry-standard DeBERTa struggles with. However, neither your Milestone 3 document nor your

Midterm Progress Report acknowledges this shortfall.

You do not need to re-train the model — but you should:

- Add a brief statement in your written analysis (and in the Midterm Progress Report if you revise it) noting that 83.31% falls slightly below the 85% F1 target.
- Explain why: the training corpus is 88% jailbreak-style adversarial data (rogue-security/prompt-injections-benchmark), which is significantly harder than the Deepset-only distribution the contract target was calibrated against.
- Note whether you plan to attempt further F1 improvement in Milestone 4 or 5 (e.g., additional epochs with adjusted hyperparameters, or whether you accept this as the baseline for comparison).

→ Actions taken: added sections in both Midterm & Milestone 3 reports addressing the shortcoming in F1 score + upcoming plan to improve this metric to meet Contract Target

2. Inference Latency — Required Metric Not Reported

The contract explicitly requires mean inference latency < 50 ms per prompt. This is a named contract metric, not a nice-to-have. It is absent from your results JSON, your written submission, and your Midterm Progress Report.

The good news: the figure is derivable from your existing training logs without re-running anything.

From your Epoch 2 validation results:

Value from Epoch 2 Logs Derived Metric

eval_runtime: 19.3916s ~883 validation samples / 19.39s ≈ 21.9 ms per prompt

~22 ms per prompt is well within the < 50 ms target. Please add this to your work:

- Add inference_latency_ms to results/custom_distilbert_metrics.json (you can compute this precisely by timing a batch inference run with `time.perf_counter()`).
- Add one sentence to your Midterm Progress Report reporting the latency figure and confirming it passes the contract target.

→ **Actions taken: modified the Testing script (evaluate_custom_model.py) to properly measure inference latency per prompt. Added Justification in Milestone 3 report on why we should use Test script results rather than Training/Validating script results. Metric is added in both Midterm & Milestone 3 reports.**

3. evaluate_custom_model.py — Reproducibility Gap

Your evaluation script loads the model from the local path `models/custom_distilbert/`, but the model

weights are hosted on Hugging Face Hub (not committed to the repository). This means a fresh clone of your repository cannot run the evaluation script without first downloading the weights manually.

The README does not currently include this step.

Please add the following to your README's Environment Setup section:

- A command to download the model weights from Hugging Face before running `evaluate_custom_model.py`
- Alternatively, modify `evaluate_custom_model.py` to accept the Hugging Face Hub model ID directly so it downloads automatically on first run.

→ **Actions taken: modified evaluate_custom_model.py to pull the trained model directly from Author's Hugging Face repo. The download is light & took a couple of minutes, the final evaluation metrics are the same as stated in Milestone 3 report before this revision. Please note: If user run script from scratch, the trained model will be auto-saved locally under "models/custom_distilbert" and user will need to replace line 23 in "src/evaluate_custom_model.py": `model=hf_model_id` with `"model=models/custom_distilbert"`**

4. Midterm Progress Report — Add a Forward-Looking Section

Your progress report accurately summarizes everything completed so far, but it contains no plan for what comes next. A midterm report should include a brief section outlining the remaining milestones, any risks you anticipate, and your timeline confidence. The contract still has three significant deliverables outstanding:

- Milestone 4 (due July 5): Embedding Vector DB classifier and LLM-as-a-Judge implementation; automated benchmarking script.
- Milestone 5 (due July 19): Exhaustive benchmarking across all models; Out-of-Distribution (OOD) evaluation against indirect injections (BIPIA dataset); draft final report.
- OOD Generalization: The contract requires testing against an unseen indirect injection dataset (BIPIA) with a target OOD F1 ≥ 0.70 . This evaluation has not been mentioned in any submission to date. You should begin planning for it now, as it requires sourcing the BIPIA dataset and designing the OOD evaluation protocol.

Please revise the Midterm Progress Report to include a Plan Forward section covering these items before your next check-in.

→ Actions taken: modified Midterm Report by added a Forward-Looking section at the bottom, addressing plans & upcoming milestones, potential risks as well as timeline confidence.